
A Unified Framework of Concepts and Chunks

Karthik Singaravadelan

Pat Langley

KSINGARA3@GATECH.EDU

PATRICK.W.LANGLEY@GMAIL.COM

Institute for the Study of Learning and Expertise, 2164 Staunton Court, Palo Alto, CA 94306 USA

Abstract

Humans combine familiar elements into novel composites whose category and behavior are inherited from their parts, segmenting a stream of words into phrases or a sequence of moves into a familiar opening. This ability is better characterized as the joint exercise of two mental abilities, categorical generalization and compositional aggregation, than as either one alone: it requires both grouping novel elements with classes they have never been observed in and combining them into larger composites the system has come to recognize. We present a postulate-driven *unified theory of concepts and chunks* that treats concept-hood and chunk-hood as two aspects of every representation, and extends Cobweb — an incremental concept-formation system — with what a compositional world demands: a vocabulary for the parts that compose an element, and processes for composing children into recognized elements and decomposing recognized elements back into their parts. A second, context-indexed hierarchy alongside Cobweb’s content-indexed one is the minimal machinery that makes these extensions coherent with the sort-and-learn cycle Cobweb already runs; the two hierarchies together support parsing, generation, and incremental learning under a single elaboration loop. We instantiate the theory in TRELLIS, an implemented system that recovers gold chunks at high precision and recall and generates near-perfectly grammatical sentences from memory on a series of synthetic context-free grammars, without supervision. We close by discussing related approaches to chunking and concept formation and outlining directions for future work.

1. Introduction

The cognitive systems literature has long sought computational accounts of how humans combine familiar parts into novel composites whose category and behavior reflect those of their parts (Langley et al., 2009). That competence is what philosophy and formal linguistics call *compositionality* (Montague, 1973): a strictly representational property, distinct from but often expressed through the hierarchical structure that repeated composition produces, and one a cognitive system must support if it is to be systematic and productive over constituent-preserving representations (Fodor & Pylyshyn, 1988). The cognitive systems community has studied this competence in two parallel paradigms (Langley, 2025): *concept formation* targets the categorical generalization that groups a novel *element* (a raw observation from outside, or a composite the system has already put together from earlier ones) with classes whose members it has never seen, while *chunking* targets the compositional aggregation that assembles familiar elements into larger recurring composites, often (but not necessarily) producing nested hierarchical structure.

Concepts are categorical, graded mental structures that summarize regularities across a set of entities and are organized taxonomically (Rosch et al., 1976; Murphy, 2002). Cobweb (Fisher, 1987; Gennari et al., 1989), the system we build on and review in Section 2, incrementally sorts attribute–value instances into a taxonomic hierarchy, but it and its variants inherit a flat attribute–value substrate with no vocabulary for naming relations between instances; the compositional structure pervading language, music, chess, and scene perception sits outside what it can express. The psychological evidence makes the gap sharper: *how* concepts combine is a substantive problem with no naive compositional answer, as the guppy effect (Osherson & Smith, 1981), pervasive over- and under-extension in conjunctions (Hampton, 1988), and the theory-driven character of combination (Murphy & Medin, 1985) all attest.

Chunks are structures that denote collections of entities forming a specified configuration, often hierarchically (Miller, 1956; Chase & Simon, 1973). Their psychological footprint is wide, from chess masters perceiving a board in familiar multi-piece configurations (Chase & Simon, 1973) to language learners aggregating phonemes into syllables, syllables into words, and words into phrases. The chunking idea itself was formalized in CHREST (Gobet & Lane, 2012), a descendant of EPAM (Richman, 1991), and runs through grammar-induction work (Wolff, 1982; Langley & Stromsten, 2000); adjacent literatures sharpened what compositional structure must support, including role–filler binding primitives (Smolensky, 1990; Plate, 1995). Chunking systems in the cognitive-systems lineage nevertheless share a limitation concept-formation systems do not: they compose freely but do not generalize across structurally similar instances. A chunk encoding a particular noun phrase cannot, by its own machinery, recognize a structurally identical one whose component words it has never seen.

The two paradigms have complementary limitations: concepts generalize but do not compose; chunks compose but do not generalize. Langley (2025) argues they have stayed disjoint for institutional rather than principled reasons, and that a unified theory should treat concept-hood and chunk-hood as two aspects of every representation. We share that diagnosis, and we take Cobweb as our starting point precisely because it already delivers the categorical half. What Cobweb lacks — and what a unified system for concepts and chunks must supply — is (i) a vocabulary for the *compositional existence* of its elements, so that a composite can name the parts that compose it in terms Cobweb can already summarize, and (ii) matched *composition and decomposition* processes that sit alongside Cobweb’s sorting-driven descent, letting the system build recognized elements out of children at recognition time and unpack recognized elements back into children at generation time. Our theory supplies both, and its central architectural move is that a second, context-indexed hierarchy alongside Cobweb’s content-indexed one is the minimal machinery that makes these two extensions coherent with Cobweb’s existing sort-and-learn cycle: every element carries a *content* description that names its parts and a *context* description that names its surroundings, the two hierarchies index the same elements through those complementary lenses, and an incremental parsing loop over both plus an inverse generation loop realize the composition and decomposition processes. We instantiate the theory in TRELLIS and evaluate it on three synthetic context-free grammars and a terminal-vocabulary experiment. The paper makes three contributions: (i) a postulate-driven *unified theory of concepts and chunks* that extends Cobweb with the two additions above, (ii) a concrete

system unifying parsing, generation, and learning under one elaboration loop, and (iii) an *empirical study* characterizing parse accuracy, generation grammaticality, and incremental absorption.

2. A Review of Cobweb

Fisher (1987)’s original Cobweb introduced an incremental, unsupervised taxonomic learner over discrete attribute–value instances. Subsequent variants extended the framework in different directions: relational extensions such as TRESTLE (MacLellan et al., 2016) handled structured instances; Cobweb/4L (MacLellan et al., 2022) refined the basic-level computation; convolutional and image-classification variants (Lian & MacLellan, 2024) extended its reach to perceptual data; and the Matsakis system (Matsakis et al., 2023) layered iterative additions on top to verify. Across all of them, the same four commitments recur: a representational distinction between instances and concepts, a single taxonomic hierarchy that organizes them, a sorting-driven performance routine, and an interleaved sorting-with-learning loop. We restate those commitments below as postulates so that the chunking theory we present in Section 3 can extend them directly.

2.1 Representation

- R1.** Knowledge takes two forms: instances and concepts.
- R2.** An instance is a set of attribute–value pairs describing a single observation.
- R3.** A concept is a probabilistic summary of a set of instances, expressed as counts over attribute–value pairs.

Concepts in Cobweb are counters rather than exemplars: each holds, for every attribute, a tally of how often each value was observed at instances sorted to it. Cobweb additionally treats attributes as conditionally independent given a concept, with each value distribution smoothed by a uniform prior. This conditional-independence assumption is what lets us route an element’s content and context descriptions through two separate hierarchies in Section 3 without changing what happens at any individual node.

2.2 Organization

Concepts in Cobweb are not stored in isolation; they are arranged in a structure that supports incremental classification.

- O1.** Concepts are organized in a taxonomic hierarchy rooted at a single concept that summarizes all observed instances.
- O2.** Each non-terminal node has two or more children that exclusively partition the instances it summarizes, and instances themselves reside as the terminal nodes.

Each non-root node refers to a strict subset of its parent’s instances, so descent corresponds to specialization rather than composition; the partition at each non-terminal is exclusive, and terminal nodes summarize singleton instances, so the same routing machinery handles intermediate abstractions and individual observations. The taxonomic shape is what our theory preserves while adding a partonomic dimension on top.

2.3 Performance

Cobweb uses the same hierarchy for two performance routines: recognition and prediction.

- P1.** An instance is sorted down the hierarchy from the root toward a terminal node, with each non-terminal routing the instance to the child concept that best fits it under a sorting criterion.
- P2.** Predictions about unobserved attributes of an instance are read from the concept to which that instance is sorted.

Cobweb’s descent is single-pass: the system commits to one child at each level and never backtracks. The sorting criterion is typically category utility, trading intra-category coherence against inter-category discriminability.

2.4 Learning

- L1.** Learning is interleaved with performance: each sorted instance is absorbed into the hierarchy along the same path that classified it.
- L2.** At each non-terminal along the path, the system selects among four actions: *add* the instance to an existing child, *create* a new child, or *restructure* the hierarchy via *merge* or *split*.

Learning is *incremental*, *piecemeal*, and *unsupervised*. The four-action menu separates two concerns: *add* and *create* grow the tree without rearranging the existing structure, while *merge* and *split* restructure it. The action selected at every step improves the same sorting criterion that routed the instance.

3. A Unified Theory of Chunking and Categorization

The Cobweb commitments of Section 2 carry directly into the unified theory we propose; the postulates below extend them, with numbering continuing from Section 2 to make the inheritance explicit. Our central claim is that Cobweb’s substrate is missing exactly two things to accommodate compositional elements: a vocabulary for the parts that compose an element, so that a composite can be summarized in the same attribute–value terms Cobweb already handles, and matched processes for composing children into recognized elements and decomposing recognized elements back into their parts. A second, context-indexed hierarchy alongside Cobweb’s content-indexed one is the minimal machinery that makes both additions coherent with Cobweb’s sorting loop, and the postulates that follow lay out the vocabulary, the organization of the two hierarchies, and the composition and decomposition processes that run over them.

3.1 Representation

The theory’s representational vocabulary starts from the same instance/concept distinction as Cobweb but adds a richer model of what an experience contains.

- R4.** An experience is a collection of instances together with a set of discrete relations between them, represented as a set of primitive elements and the local relations that arrange them in the experience.

- R5.** Every element carries two descriptions: a *content* description, which names the parts that compose it, and a *context* description, which names the discrete relations it bears to surrounding elements.
- R6.** Every element is either a primitive or a composite; a composite (also called a *chunk*) is an element built from other primitive or composite elements, and its content description names those parts in the same vocabulary the rest of the system uses.

The two descriptions let the same element be recognized through two independent categorical lenses: content bears on composition (a composite names its children, a primitive names nothing), while context bears on distribution (the sliding window of neighbours, taken over boundary positions for composites). Keeping them separate is the discrete reading of the structure/content factorization that runs through binding-based accounts (Smolensky, 1990): a single bundle would force role and filler into one signal, whereas two indexed descriptions keep them from interfering.

3.2 Organization

Two structures hold the elements together: one records what was built, while the other summarizes the categories the parts fall under.

- O3.** Primitives and composites are jointly organized in a partonomic hierarchy that records, for each composite, the elements it was built from.
- O4.** The system maintains long-term memory in the form of two distinct taxonomic hierarchies: a *content hierarchy* that organizes the content descriptions of R5, and a *context hierarchy* that organizes the context descriptions of R5.

The partonomic hierarchy of O3 holds a parse together and answers “what are the parts?”; the taxonomic hierarchies of O4 answer “what category does this fall under?”. The two-hierarchy commitment is the central organizational claim of the theory: a context-tree path groups noun-like tokens together regardless of whether they appeared as primitives or composites, while a content-tree path groups Det+N-shaped composites together regardless of what surrounded them, and neither signal leaks into the other.

3.3 Performance

The performance loop runs in two directions: parsing builds composites from primitives, generation expands them back.

Parsing.

Parsing runs bottom-up: it repeatedly proposes composites over the primitives and composites currently on the frontier, commits the most recognizable one, and grows a partonomic tree until no further composite is recognized. The five postulates below fix what a candidate is, how it is judged, and when the process halts.

- P3.** Parsing is an incremental process of elaborations on an input experience.
- P4.** A single elaboration generates a set of candidate chunks consistent with the current partial parse.

- P5.** Each candidate chunk is scored against the content and context hierarchies and gated by a recognition threshold.
- P6.** The best-scoring candidate that passes the recognition threshold is committed to the partonomic parse tree as a new composite element.
- P7.** Parsing terminates when no remaining candidate passes the recognition threshold.

The content score asks whether the candidate’s composition looks like one the system has consolidated before; the context score asks whether its surroundings look like ones that have hosted similar composites. P7 lets the system fail gracefully: when nothing clears the threshold, the parse ends and the remaining forest is returned rather than spurious chunks being asserted at the boundary of competence.

Generation.

Generation runs the same loop in reverse: it starts from a recognized seed and repeatedly expands it into children, descending the partonomy until only primitives remain. The postulates mirror those for parsing — an expansion step, what a candidate expansion is, how it is sampled, what commits, and when the process halts.

- P8.** Generation is an incremental process of expansions on a composite seed.
- P9.** A single expansion produces a candidate leaf consistent with the basic-level concept above the seed’s content reference.
- P10.** The candidate leaf is sampled from the content hierarchy and conditioned on the context hierarchy.
- P11.** The sampled leaf is committed to the partonomic generation tree as the seed’s expansion, and its content children (if any) become new seeds.
- P12.** Generation terminates when every expanded element is a primitive.

The basic-level constraint of P9 plays the same role for generation that the recognition threshold plays for parsing: it anchors sampling at a well-supported level of abstraction rather than at an idiosyncratic deep leaf (Rosch et al., 1976). Conditioning the sampled leaf on the context hierarchy (P10) keeps generation distributionally consistent with what the system has seen.

3.4 Learning

Learning in the theory routes each commit through both hierarchies, but the routing differs by element type.

- L3.** Learning consists of incorporating the elaborations produced by parsing into the two taxonomic hierarchies.
- L4.** Each composite committed during parsing is added to both hierarchies (its content instance to the content hierarchy and its context instance to the context hierarchy), while primitives, which carry no content instance, are added to the context hierarchy alone.
- L5.** Each addition follows the standard sorting-with-learning procedure of the taxonomic hierarchy that receives it.

Every commit is followed immediately by an update to memory, applying Cobweb’s L1 interleaving commitment to the two-hierarchy substrate. The asymmetric routing of L4 makes the content hierarchy a record of actually-composed structure rather than surface vocabulary, since primitives contribute only to the context hierarchy. L3–L5 together make recognition and learning two faces of a single elaboration step.

4. The TRELLIS Chunking System

TRELLIS instantiates the theory of Section 3 atop the Cobweb framework of Section 2. We describe its representation, memory organization, parsing and generation processes, and the way it absorbs results back into memory; exact formulas appear in Appendix C.

4.1 Representation

Every element in TRELLIS carries two attribute–value descriptions: a *content instance* and a *context instance*. The content instance names the parts that compose the element; the context instance names the neighbours that surround it. Each is routed to its own hierarchy in long-term memory. Primitives are atomic and carry only a context instance; composites carry both, as required by R5. A composite’s content instance names its left and right children by two kinds of attribute: an identifier pointing into the context hierarchy (where each child has already been classified), which says *what* the child is, and a *compositional complexity* tag recording the depth of the subtree it spans, which says how deeply nested it is. Figure 1 illustrates this two-instance representation for a sample noun-phrase chunk.

Bag-of-concepts encoding. The value associated with every attribute is not a single symbol but a small bag (a counter) over concept identifiers, weighted by position: a primitive’s context attributes hold bags of its neighbours’ concept identifiers, and a composite’s content attributes hold bags of its children’s context identifiers. The rationale is that Cobweb concepts are *predictive but mixed* — a concept’s count distribution summarizes every instance that descended through it, so the single most-likely value throws away the alternatives the node is genuinely uncertain between. Keeping the bag preserves that structure and lets the downstream hierarchy compose parts that straddle a category boundary without first being forced to commit, while keeping the representation uniform across primitives, composites, and depths of the partonomy (R3). The complexity tags are the structural counterpart: chunks whose children share a distributional context but differ in nesting depth share a bag signature yet differ in complexity, so keeping complexity *visible* lets category utility separate NP=DET+N from S=NP+VP without any category label.

Content-reference remapping. Because the context hierarchy restructures as new instances are absorbed, content references committed at training time t_1 may dangle by t_2 . TRELLIS handles this with a top- k ancestor pool: each content-reference is canonicalized to a recent ancestor of its target leaf, and whenever the context tree restructures the encoder pushes a fresh value-remap binding so the corresponding bag entries in every stored content instance are rewritten on the fly to the new canonical identifier. Without the remap, the content hierarchy would gradually accumulate dead

references to context-tree concepts that no longer exist; the pool depth and the top- k width are implementation parameters whose values appear in Section 5.1.

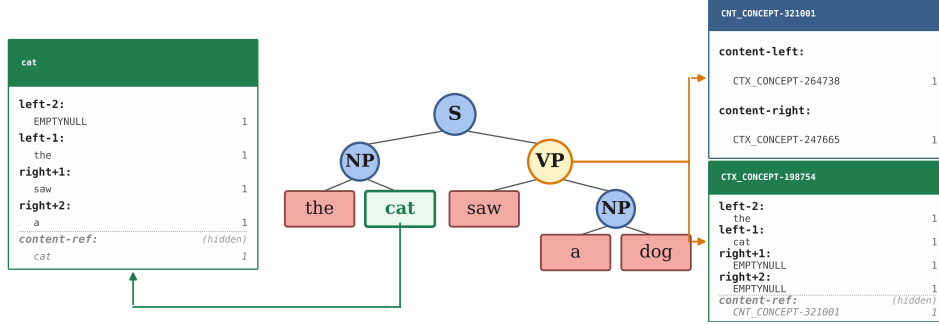


Figure 1. Parse of “the cat saw a dog” showing a primitive (“cat”, green) and a composite (“saw a dog”, amber). Each box header is the long-term-memory identifier under which the instance is classified: a primitive’s classifier is the surface word itself, while the composite carries a content-tree identifier (CNT_CONCEPT-*) and a context-tree identifier (CTX_CONCEPT-*). Each context instance has slot attributes (left-2, left-1, right+1, right+2, shown truncated to two positions per side for readability; the experiments use five, i.e. ten slots) plus a content-ref row (dashed italic) carried as metadata but ignored during context-hierarchy routing.

4.2 Organization

TRELLIS organizes long-term memory into two Cobweb taxonomies, as required by O4. The content hierarchy classifies the content instances of composites: because content references are drawn from the context hierarchy, composites with structurally similar children occupy nearby positions, reflecting their shared role as NP-filling constituents. The context hierarchy classifies every element by its surroundings, so a noun and an adjective phrase end up sorted together when they appear before and after the same kinds of neighbours. Figure 2 shows leaf-level subtrees of each. Alongside these two long-term structures sits a transient scratchpad: the partonomic parse tree for the current experience.

4.3 Performance

TRELLIS realizes P3–P12 as two complementary processes: parsing builds chunks incrementally from an experience, and generation expands chunks back into primitives.

4.3.1 Parsing.

A parse is initialized with the input’s primitives, each admitted to the frontier only when its context-hierarchy path clears the climbing-ancestor gate described below. Each elaboration then forms a candidate chunk for every adjacent pair on the frontier and routes its content and context instances through their hierarchies, scoring each by the class-posterior log-probability $\log P(C \mid \mathbf{i})$ — the discriminative, EPMI-style recognition score (Appendix C). The same climbing-ancestor walk on

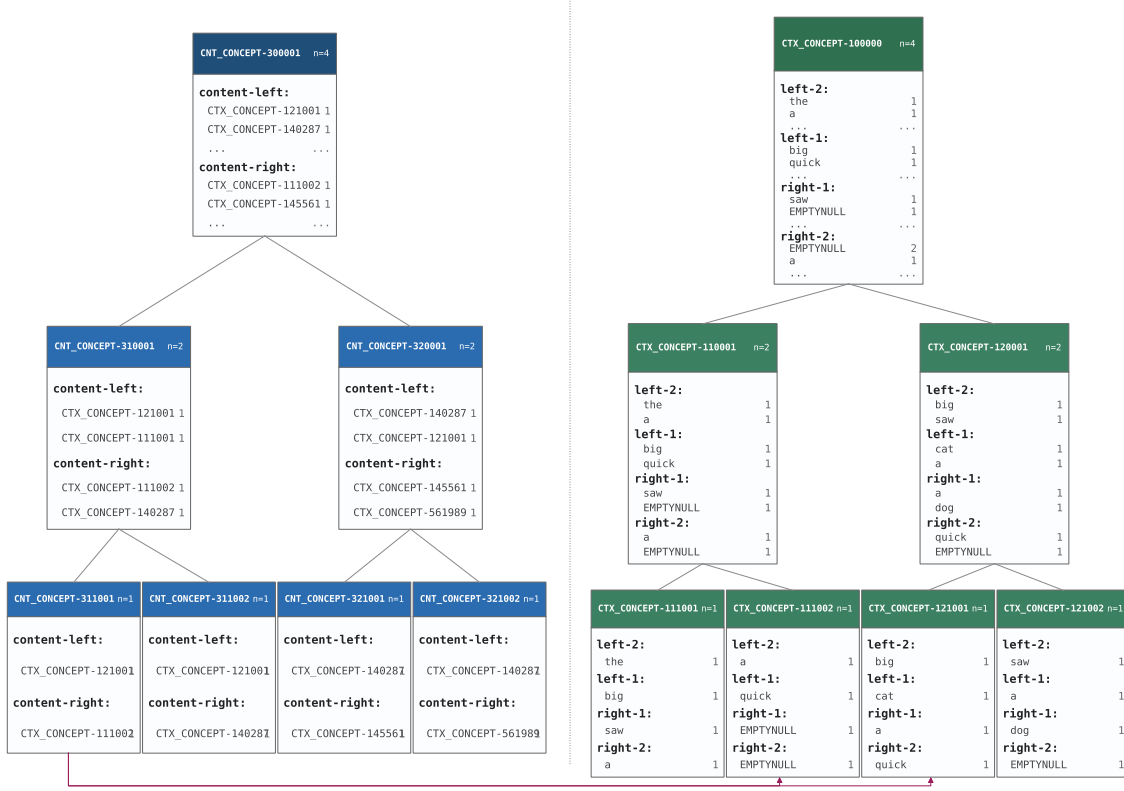


Figure 2. Leaf-level subtrees of the content (left) and context (right) hierarchies. Each node is a Cobweb concept with instance count n . Content-tree attributes `content-left/right` take `CTX_CONCEPT-*` values pointing into context-tree leaves; context-tree attributes `left/right-1, -2` take terminal words, and higher-level concepts sum the distributions below them, truncated to the top two values with “...” indicating further entries. Arrows show the cross-hierarchy reference: a content leaf’s child slots name the context concepts that classify its children.

the context path supplies the threshold: a candidate passes when the walk finds a non-root level with support above a tuned minimum and fails when it reaches the root. Failed candidates are discarded; the highest-scoring survivor is committed, its children leave the frontier, the composite takes their place, and the next elaboration runs against a memory updated as in Section 4.4. The loop terminates when no candidate clears the threshold; primitives and composites share the same gate, so one notion of “mature representation” applies to both. Figure 3 summarizes one round.

4.3.2 Generation.

Generation inverts the parsing loop. A generation step begins with a seed (a leaf identifier in the content hierarchy carried by an unexpanded composite). TRELLIS walks the seed upward to a *well-supported* ancestor — the shallowest whose accumulated count clears the maturity threshold, the same climbing-ancestor notion the parsing gate uses — and recombines from the pool of training

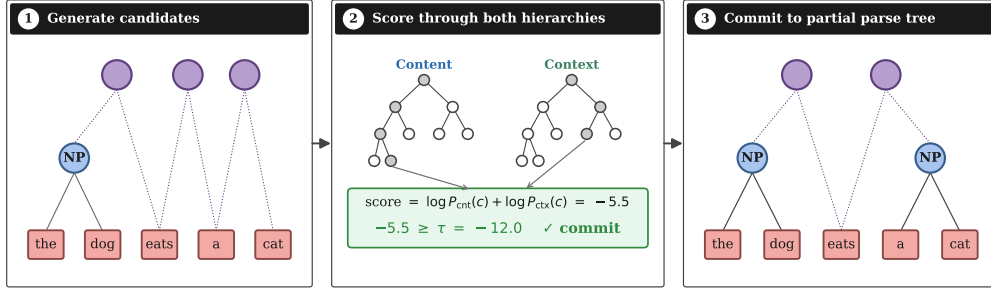


Figure 3. One parsing elaboration: (1) form candidates on the frontier (purple); (2) route each through both hierarchies; (3) commit the best one clearing the threshold.

chunks recorded at that level, sampling each of the two child slots conditioned on the context-hierarchy class the parent expects for it; it then recurses on the sampled children. The recursion terminates when every sampled leaf is a primitive. Anchoring at a well-supported level rather than a deep leaf plays the same role for generation that the recognition threshold plays for parsing; conditioning each child on its context class keeps the mixed intermediary level from emitting a wrong-type constituent, so the pool stays grammatical while still reaching combinations no single training leaf could produce (Appendix C). Figure 4 summarizes one expansion end-to-end.

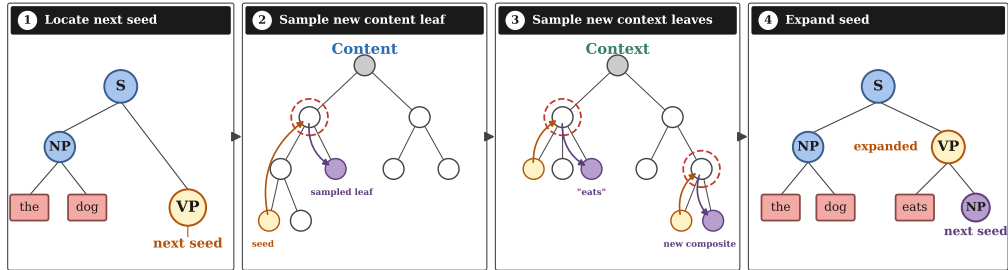


Figure 4. One generation expansion: (1) locate the next seed; (2) walk up to its well-supported maturity level (dashed red) in the content tree and sample a fresh chunk (purple); (3) repeat for each of its two child references, conditioning each on the context class the parent expects; (4) attach the results to the seed.

4.4 Learning

Every commit during the parsing loop is followed immediately by an update to memory, so parsing and learning interleave one step at a time (L3). TRELLIS sorts a committed composite’s content instance into the content hierarchy and its context instance into the context hierarchy via the standard Cobweb procedure (L2); the two sorts run independently. Primitives are sorted into the context hierarchy alone, since they carry no content instance. Restricting primitives to the context hierarchy keeps the content hierarchy compositional by construction: every leaf there is a composite whose children are valid context references, which is exactly the invariant the generation procedure relies

on. Because both hierarchies update immediately, candidates produced later in the same parse are scored against a memory that already reflects the chunks committed earlier.

5. Evaluation of TRELLIS

We evaluate TRELLIS along two axes of dataset complexity: varying the number of non-terminal categories while holding the lexicon fixed (Section 5.2) stresses the categorical dimension, while varying the terminal-lexicon size under a fixed grammar (Section 5.3) stresses the distributional dimension by thinning per-word support. Together the two experiments probe how much of each hierarchy’s work is doing which job. Grammars, lexicons, and one ground-truth example parse per condition are collected in Appendix A.

5.1 Setup

Grammars and training. All three grammars are strictly binary: every syntactic non-terminal expansion has exactly two right-hand-side elements (lexical preterminals retain their unary $POS \rightarrow word$ rewrites). The SMALL grammar admits a determiner-noun NP and a transitive VP; MED adds a recursive AdjP head, prepositional phrases, and a packaged direct object plus PP through an intermediate VPobj; LARGE adds relative-clause attachment via a separate Nbar non-terminal, keeping AdjP semantically faithful to adjective recursion alone. The three are strictly nested ($SMALL \subset MED \subset LARGE$) in both productions and terminals; Appendix A lists the complete productions and lexicons. For each grammar we drew a 400-sentence derivation-annotated corpus, trained TRELLIS incrementally on 320 sentences, and evaluated on the 80-sentence held-out remainder. All metrics are five-seed means with a ± 1 standard-deviation band.

System parameters. Two tuned parameters govern the system’s behavior: the uniform Cobweb prior α (R3), which controls base-rate smoothing in every per-attribute value distribution, and the recognition threshold τ , parameterized as a minimum context-hierarchy count, which controls how readily a candidate is judged familiar; their exact forms appear in Appendix C. A single parameter set is used unless explicitly noted: context window size 5 tokens per side (ten positional slots), content-tree Cobweb prior $\alpha_{\text{content}} = 10^{-4}$ and context-tree prior $\alpha_{\text{context}} = 10^{-5}$ (a sharper context prior makes the two hierarchies cleanly separable), basic-level prior $\alpha_{\text{bl}} = 10$, and recognition threshold $\tau = 30$.

Metrics. We track two complementary axes. *Omission* concerns whether the parser recovers the gold constituent structure of held-out sentences; we report it as bracket F1, where a fully correct parse scores 1 and every missed or spurious bracket lowers the score. *Commission* concerns whether the generator stays inside the source grammar; we report it as generation grammaticality, the fraction of sampled sentences the grammar accepts under CKY recognition. The omission/commission decomposition is due to Langley & Stromsten (2000) for GRIDS.

5.2 Scaling Behavior Across Grammar Complexity

Figure 5 plots the omission (left, held-out parse F1) and commission (right, generation grammaticality) overlay across the three grammars as the training corpus grows. Omission resolves quickly on SMALL and more slowly on MED and LARGE, the F1 curves ordering monotonically by complexity (converging to 1.00, 0.96, 0.95). Generation grammaticality reaches 0.98–1.00 on every grammar; the residual few-percent commission traces to child slots where the maturity-level pool holds no context-compatible chunk, forcing a fallback that can admit a wrong-type constituent (per-condition five-seed scores in Appendix B). The asymmetry is consistent with the postulates: parsing depends on per-candidate threshold judgements that need not improve monotonically as the content hierarchy restructures, while generation depends only on the structural invariants L4 and L5 keep well-formed by construction.

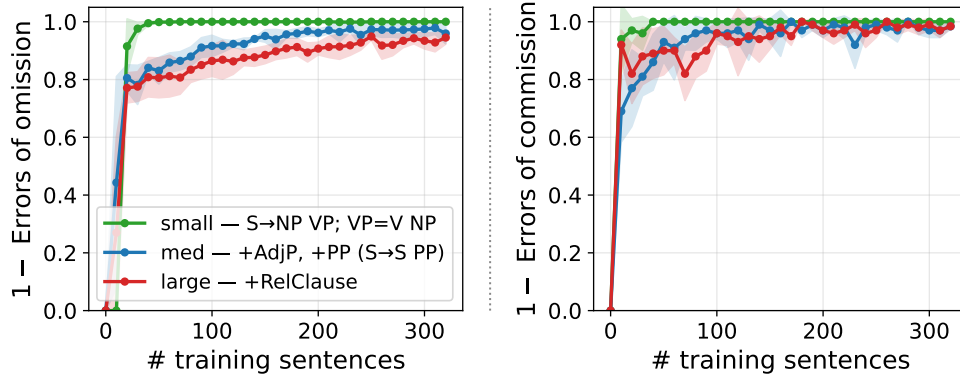


Figure 5. Omission (left, held-out parse F1) / commission (right, generation grammaticality) overlay across the three grammars as the training corpus grows (5 seeds per grammar, $\pm 1\sigma$ band). The terminal vocabulary is held constant; only the grammar varies. Parse F1 converges within ~ 100 sentences, grammaticality by ~ 300 .

5.3 Scaling Behavior Across Terminal Vocabulary Size

Figure 6 repeats the overlay across three terminal-vocabulary sizes (LOW: 11, MED: 22, HIGH: 39 terminals) on the MED grammar, holding the productions fixed and varying only the lexicon size per part-of-speech category. The two-hierarchy organization absorbs per-terminal sparsity by construction: the context hierarchy groups distributional equivalents (nouns with nouns, verbs with verbs), so each terminal inherits its part-of-speech support from its POS siblings regardless of how often the surface form itself was observed. The content hierarchy in turn sees no growth in its attribute vocabulary, since composites continue to refer to children by context-hierarchy identifier rather than by surface word.

The same climbing-ancestor gate is used unchanged across the three variants: a larger lexicon simply requires deeper descent before a terminal’s context-tree path clears the threshold, which the procedure supplies automatically as the context hierarchy grows. All three converge to strong scores

on both axes — parse F1 around 0.96, 0.95, 0.91 for LOW/MED/HIGH and generation grammaticality around 0.91, 0.96, 0.96 (per-condition five-seed scores in Appendix B).

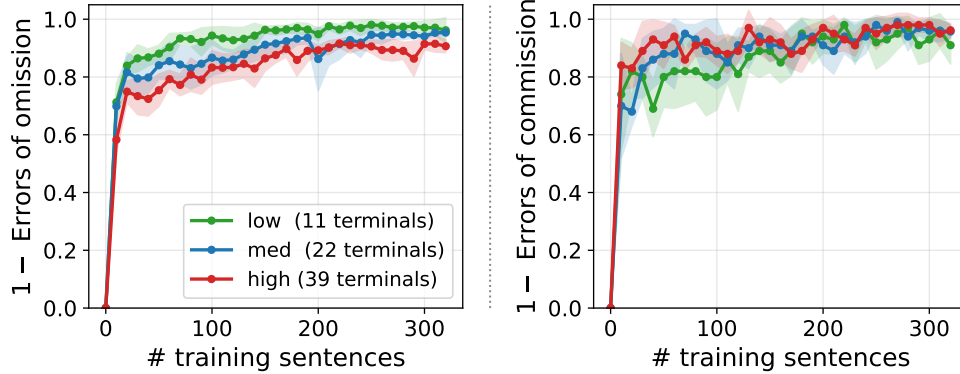


Figure 6. Omission (left, held-out parse F1) / commission (right, generation grammaticality) overlay across three terminal-vocabulary sizes on the MED grammar (5 seeds per size, $\pm 1\sigma$ band). The grammar is held constant; only the lexicon size per POS varies. Generation grammaticality converges by ~ 300 to 0.96 (MED, HIGH) but only 0.91 for the sparsest lexicon (LOW), which fills its recombination pools slowest.

6. Related Work

Several bodies of work cover some but not all of what TRELIS delivers: two-hierarchy categorical/compositional memory, incremental parsing and inverse generation under a single learning loop, and unsupervised acquisition of constituent structure. We organize this section by what each line of work omits.

6.1 Unified Theories of Concepts and Chunks

The closest precursor is Langley (2025)’s essay arguing that a unified theory should treat concept-hood and chunk-hood as two aspects of every representation. The essay proposes representational, organizational, performance, and learning commitments any unified system would need to satisfy, but does not commit to an implementation; our theory is one concrete instance of it. Earlier work in the same direction includes the Matsakis system (Matsakis et al., 2023), which extends Cobweb with iterative script-like additions but lacks the content–context distinction that keeps compositional and distributional structure separable in our framework, and the convolutional Cobweb variant (Lian & MacLellan, 2024), which pairs a chunking-style construction process with a classification-style hierarchy but fixes the construction by architecture and operates on images rather than the sequential partonomic structure that drives our parsing loop. The CHREST family (Gobet & Lane, 2012; Richman, 1991) sits adjacent to this work in modeling expert chunk acquisition over discrimination networks, but its chunks match input by exact lookup rather than generalize across structurally similar instances; this is the limitation we identified in Section 1 and the limitation the two-hierarchy organization is designed to lift.

6.2 Grammar Induction

The problem of inducing a grammar from unannotated sentences asks for the same combination of compositional structure and categorical generalization our theory addresses. Wolff (1982) introduced an early system that merges frequent adjacent pairs and generalizes the resulting categories; its two operations prefigure our two hierarchies, though they are not maintained as separate structures. Langley & Stromsten (2000) introduced GRIDS, which frames grammar induction as Bayesian model selection and evaluates candidates by the omission/commission decomposition we adopt in Sections 5.2 and 5.3. These systems target the same competence TRELLIS targets but treat the categorical and compositional aspects as a single optimization rather than as two separate dimensions of memory, and they correspondingly lack a single substrate from which both parsing and generation fall out under one elaboration loop.

6.3 Distributional and Neural Representations

A third body of work shows that the local context in which a token appears can drive surprisingly rich syntactic categorization. Distributional embeddings cluster words that occur in similar contexts, and probes can recover part-of-speech tags and constituency structure from intermediate layers of contextualized models. These models extract a context-driven categorical signal of the kind our context hierarchy maintains, but their structure is implicit in trained parameters rather than read off a discrete substrate, which limits the postulate-level analysis a cognitive system can perform on it. Our theory is a discrete counterpart that keeps the context-driven categorical signal explicit and pairs it with a content-driven one of the same kind, making composition explicit rather than an inferred property of the representation.

7. Concluding Remarks

We have presented a postulate-driven theory of chunking that combines the categorical commitments of concept formation with the compositional commitments of the chunking tradition: every element carries both a content and a context description, organized across two Cobweb hierarchies, with one elaboration loop turning them into parsing, generation, and incremental learning. TRELLIS recovers gold chunks at high precision and recall and generates near-perfectly grammatical sentences from memory, without supervision. One caveat to the implementation is that the basic-level concept is itself an active area of study, and our count-based maturity cut identifies the well-supported level of abstraction heuristically rather than against an independently verified definition.

The unified theory places chunks and contexts in a mutual dependence neither can do without: chunks are identifiable only because their contexts are similar, and contexts are informative only because they constrain the chunks that fill them. This dependence is also where the empirical asymmetry between parsing and generation finds its natural explanation: parsing fails when the context-hierarchy support for a candidate is not yet strong enough to clear the recognition threshold, while generation succeeds because every chunk in the content hierarchy was placed there by a parse that had already cleared it. The two procedures are inverses of each other but draw on the two

hierarchies asymmetrically, and the asymmetry is what gives the system its graceful-failure behavior under threshold tuning.

Two future endeavors follow. *Unsupervised chunk learning* is the most pressing: TRELLIS still relies on a tuned recognition threshold, and earlier statistical approaches (MDL grammars, mutual-information chunkers, frequency-driven mergers) struggled to say what makes a chunk *good* once superficial recurrence had been exhausted. The theory points toward a chunk-intrinsic answer: a chunk is good to the extent that it is *stable* across the contexts in which it appears, *reusable* as a part of larger chunks, and *predictive* of its own neighbours. An adaptive criterion tracking memory growth should be able to identify such chunks without manual tuning. The second endeavor is to push the substrate *beyond sentences* along macro and micro axes: upward into paragraphs and discourse units, downward into letters and phonemes, with the basic-level frontier shifting as more abstract chunks accumulate. The single left/right discrete relation used in TRELLIS should likewise generalize to a broader vocabulary of discrete relations (gridlike neighbourhoods, compass directions, temporal sequences) without changing the rest of the postulates. The unification is one the cognitive systems literature has gestured toward for decades but rarely committed to in implementation; the theory and TRELLIS together establish it as a viable foundation.

Acknowledgements

We would like to thank Chris MacLellan, Zekun Wang, Xin Lian, Kyle Moore, and friends at the Teachable AI Lab for discussions.

References

- Chase, W. G., & Simon, H. A. (1973). Perception in chess. *Cognitive Psychology*, 4, 55–81.
- Fisher, D. H. (1987). Knowledge acquisition via incremental conceptual clustering. *Machine Learning*, 2, 139–172.
- Fodor, J. A., & Pylyshyn, Z. W. (1988). Connectionism and cognitive architecture: A critical analysis. *Cognition*, 28, 3–71.
- Gennari, J. H., Langley, P., & Fisher, D. H. (1989). Models of incremental concept formation. In J. G. Carbonell (Ed.), *Machine learning: Paradigms and methods*, 11–61. Cambridge, MA: MIT Press.
- Gobet, F., & Lane, P. C. R. (2012). CHREST: A model of perception, learning, and problem solving. In N. M. Seel (Ed.), *Encyclopedia of the sciences of learning*, 556–559. New York, NY: Springer.
- Hampton, J. A. (1988). Overextension of conjunctive concepts: Evidence for a unitary model of concept typicality and class inclusion. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 14, 12–32.
- Langley, P. (2025). Concepts and chunks in cognitive systems. *Advances in Cognitive Systems*, 11, 1–12.
- Langley, P., Laird, J. E., & Rogers, S. (2009). Cognitive architectures: Research issues and challenges. *Cognitive Systems Research*, 10, 141–160.

- Langley, P., & Stromsten, S. (2000). Learning context-free grammars with a simplicity bias. *Proceedings of the Eleventh European Conference on Machine Learning* (pp. 220–228). Barcelona, Spain: Springer.
- Lian, X., & MacLellan, C. J. (2024). Convolutional Cobweb: A new approach to image classification through concept formation. *Advances in Cognitive Systems* (pp. 221–238).
- MacLellan, C. J., Harpstead, E., Aleven, V., & Koedinger, K. R. (2016). TRESTLE: A model of concept formation in structured domains. *Advances in Cognitive Systems* (pp. 131–150).
- MacLellan, C. J., Matsakis, N., & Langley, P. (2022). Efficient induction of language models via probabilistic concept formation. *Advances in Cognitive Systems* (pp. 1–16).
- Matsakis, N., MacLellan, C. J., & Langley, P. (2023). Incremental induction of phrase-structure grammars through probabilistic concept formation. *Proceedings of the Eleventh Annual Conference on Advances in Cognitive Systems*. Atlanta, GA.
- Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63, 81–97.
- Montague, R. (1973). The proper treatment of quantification in ordinary English. In K. J. J. Hintikka, J. M. E. Moravcsik, & P. Suppes (Eds.), *Approaches to natural language*, 221–242. Dordrecht: Reidel.
- Murphy, G. L. (2002). *The big book of concepts*. Cambridge, MA: MIT Press.
- Murphy, G. L., & Medin, D. L. (1985). The role of theories in conceptual coherence. In *Psychological review*, volume 92, 289–316.
- Osherson, D. N., & Smith, E. E. (1981). On the adequacy of prototype theory as a theory of concepts. *Cognition*, 9, 35–58.
- Plate, T. A. (1995). Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6, 623–641.
- Richman, H. B. (1991). *Discrimination net models of expertise*. Doctoral dissertation, Department of Psychology, Carnegie Mellon University, Pittsburgh, PA.
- Rosch, E., Mervis, C. B., Gray, W. D., Johnson, D. M., & Boyes-Braem, P. (1976). Basic objects in natural categories. *Cognitive Psychology*, 8, 382–439.
- Smolensky, P. (1990). Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence*, 46, 159–216.
- Wolff, J. G. (1982). Language acquisition, data compression, and generalization. *Language and Communication* (pp. 57–89).

Appendix A. The Synthetic Grammars

This appendix lists the productions and terminal lexicons of the three synthetic context-free grammars used in Section 5. Every non-terminal expansion has one or two right-hand-side elements, matching the strictly binary representation used by the parser. Sentences are derived by selecting uniformly at random from the right-hand-side alternatives. The three grammars are strictly nested: every production and terminal of SMALL is retained in MED, and every production and terminal of MED is retained in LARGE.

SMALL. A determiner-noun NP and a transitive VP, sharing MED’s full determiner, noun, and verb lexicons (only the Adj and P classes and the modifier productions are absent).

S	→	NP VP
NP	→	Det N
VP	→	V NP
Det	→	the a
N	→	cat dog man woman park telescope
V	→	saw liked chased found admired

MED. Extends SMALL with adjective recursion through an AdjP head that always carries at least one adjective, prepositional phrases, and a packaged direct object plus PP through the intermediate VPobj; introduces the Adj and P classes (additions over SMALL **bolded** below). Bare-noun NPs use the no-modifier shortcut “NP → Det N” so the grammar never produces a unary syntactic node.

S	→	NP VP
NP	→	Det N Det AdjP
AdjP	→	Adj N Adj AdjP
VP	→	V NP V VPobj
VPobj	→	NP PP
PP	→	P NP
Det	→	the a
N	→	cat dog man woman park telescope
Adj	→	big small red quick lazy
V	→	saw liked chased found admired
P	→	with in on under

LARGE. Adds relative clauses on top of MED: a relative-clause NP shape (Det Nbar), the Nbar and RelClause non-terminals, and the RelPro part-of-speech class, together with a handful of extra terminals (additions over MED **bolded** below). Splitting AdjP (adjective-only) from Nbar (head with relative clause) keeps each label semantically faithful; recursion is weighted toward the non-recursive alternatives so RelClause sentences do not dominate the corpus.

S	→	NP VP
NP	→	Det N Det AdjP Det Nbar
AdjP	→	Adj N Adj AdjP
Nbar	→	N RelClause AdjP RelClause
VP	→	V NP V VPobj
VPobj	→	NP PP
RelClause	→	RelPro VP
PP	→	P NP
Det	→	the a this that
N	→	cat dog man woman park telescope boy girl teacher
Adj	→	big small red quick lazy tall curious
RelPro	→	who that which
V	→	saw liked chased found admired carried read
P	→	with in on under near

Terminal-experiment lexicons. The terminal-vocabulary experiment (Section 5.3) holds the MED productions fixed and varies only the lexicon, drawing the first k words of a shared pool for each part-of-speech class. The three variants are strictly nested ($\text{LOW} \subset \text{MED} \subset \text{HIGH}$):

POS	LOW (11)	MED (22)	HIGH (39)
Det	the, a	the, a	the, a, this, that
N	cat, dog, man	cat, dog, man, woman, park, telescope	cat, dog, man, woman, park, telescope, robot, apple, book, girl, boy, bird
Adj	big, small	big, small, red, quick, lazy	big, small, red, quick, lazy, ancient, blue, tall, wise
V	saw, liked	saw, liked, chased, found, admired	saw, liked, chased, found, admired, carried, read, knew
P	with, in	with, in, on, under	with, in, on, under, near, behind

Appendix B. Final-Checkpoint Scores

The two tables below give the five-seed parse F1 and generation grammaticality at the final training checkpoint (320 sentences), summarized anecdotally in Sections 5.2 and 5.3. Grammaticality is estimated from 500 generated sentences per seed at the endpoint, so the reported values are tight rather than the noisier per-checkpoint estimates plotted in the learning curves. The same parser, gate, and parameters are used in every condition; no per-corpus tuning.

Table 1. Grammar experiment: five-seed final-checkpoint scores across the three nested grammars, each trained on 320 sentences.

Grammar	Parse F1	Grammaticality
SMALL	100.0 $\pm$ 0.0%	100.0 $\pm$ 0.0%
MED	95.9 $\pm$ 3.6%	98.4 $\pm$ 1.0%
LARGE	94.5 $\pm$ 2.3%	98.4 $\pm$ 0.6%

Table 2. Terminal-vocabulary experiment: five-seed final-checkpoint scores across the three lexicon variants of the MED grammar.

Lexicon (POS counts)	Parse F1	Grammaticality
LOW (11 terminals)	96.2 $\pm$ 4.3%	91.0 $\pm$ 6.5%
MED (22 terminals)	95.4 $\pm$ 1.0%	95.7 $\pm$ 2.4%
HIGH (39 terminals)	90.7 $\pm$ 2.2%	96.2 $\pm$ 2.4%

Appendix C. Implementation Formulas

Recognition score. The Cobweb log-probability of an instance $\mathbf{i}$ under a concept C is, by R3,

$$\log P(\mathbf{i} \mid C) = \sum_{(a,v) \in \mathbf{i}} \log P(v \mid C, a), \quad P(v \mid C, a) = \frac{n_{C,a,v} + \alpha_a}{n_{C,a} + V_a \alpha_a},$$

where $n_{C,a,v}$ is the count of value v for attribute a at concept C , V_a is the value vocabulary size, and α_a is the uniform-prior smoothing (R3; the per-tree and basic-level values are those listed under System parameters in Section 5.1). The recognition score used to rank candidates is not this raw generative log-probability but the *class posterior* it induces by Bayes’ rule: sorting an instance $\mathbf{i}$ into a hierarchy selects a class C , and the score is $\log P(C \mid \mathbf{i}) = \log P(\mathbf{i} \mid C) + \log P(C) - \log P(\mathbf{i})$. Normalizing out the instance marginal $\log P(\mathbf{i})$ is what makes the score discriminative (EPMI-style): a dense cluster inflates the raw log-probability of everything routed through it, whereas the posterior rewards a candidate only to the extent it belongs to its class more confidently than to the alternatives, so a non-constituent that fits nowhere in particular scores low. Content and context recognition scores are the two class posteriors $s_c = \log P(C_c \mid \mathbf{i}_c)$, $s_x = \log P(C_x \mid \mathbf{i}_x)$; the ranking key is $s_x + s_c$, with s_x breaking ties.

Recognition threshold (climbing ancestor). An element’s path $\pi = (n_0, n_1, \dots, n_L)$ in the context hierarchy — whether the element is a primitive being routed to the parse frontier or a composite candidate being evaluated for commit — is walked from n_L upward to find the most-specific ancestor whose support exceeds τ :

$$\text{support}(n) = \begin{cases} n^{\text{count}} & \tau \geq 1 \text{ (absolute)} \\ n^{\text{count}}/n_0^{\text{count}} & 0 < \tau < 1 \text{ (relative)} \end{cases}$$

The element passes when some non-root ancestor has accumulated enough support; otherwise it fails. The relative mode keeps the gate’s selectivity constant as the tree grows. Sharing this gate between primitives and composites is what gives the parsing loop a single notion of “mature representation” across both layers.

Generation anchor (maturity cut). The well-supported level P9 samples from is realized by the same climbing-ancestor notion the parsing gate uses, rather than by an EPMI prediction-collapse estimate: the anchor above a leaf ℓ is the shallowest ancestor whose accumulated count clears the maturity threshold τ . This is a *maturity cut* — one upward walk, linear in tree depth. Empirically it is the level at which the substitution class stabilizes: content-hierarchy leaves are near-singletons, and the max-EPMI basic level one step further up collapses distinct constituent types together (it mixes PP, NP, and VP), so a fixed count-based cut is both cheaper and cleaner than the EPMI estimate for anchoring generation.

Generation by context-conditioned chunk replay. During training, every committed chunk is recorded as a tuple (parent-leaf, left-child, right-child), each child annotated with its own context-hierarchy class, and indexed *both* by its parent-leaf identifier and by that leaf’s maturity-cut ancestor. Generation samples a sentence-root chunk as the seed ℓ_0 and recurses on each child: a word is emitted directly; a composite child is expanded by sampling a chunk from its *maturity-level* pool $\text{pool}[\text{mat}(\ell)]$, restricted to chunks whose own context class matches the class the parent expects for that slot (the P10 conditioning), falling back to the full leaf pool when no context-compatible chunk exists. The maturity-level pool makes recombinations no single leaf could produce reachable (novelty); the context-class restriction keeps that wider pool from emitting wrong-type constituents (grammaticality). Together they yield near-perfect from-scratch grammaticality *with* a substantial fraction of outputs never seen verbatim in training.